

Lab 6: Implementation of Disk Scheduling Algorithms (FCFS, SSTF, SCAN, CSCAN, LOOK, CLOOK)

Theory:

Disk scheduling is the process used by an operating system to determine the order in which pending disk I/O requests are serviced. In a magnetic disk, the read/write head must move to the track containing the required data. The movement of the disk head contributes to seek time, so an appropriate scheduling method can reduce unnecessary movement and improve disk access performance.

A disk scheduling algorithm arranges the pending requests according to a particular strategy. The total head movement can be calculated using:

$$\text{Total Head Movement} = \Sigma |\text{Current Head Position} - \text{Next Head Position}|$$

Importance of Disk Scheduling

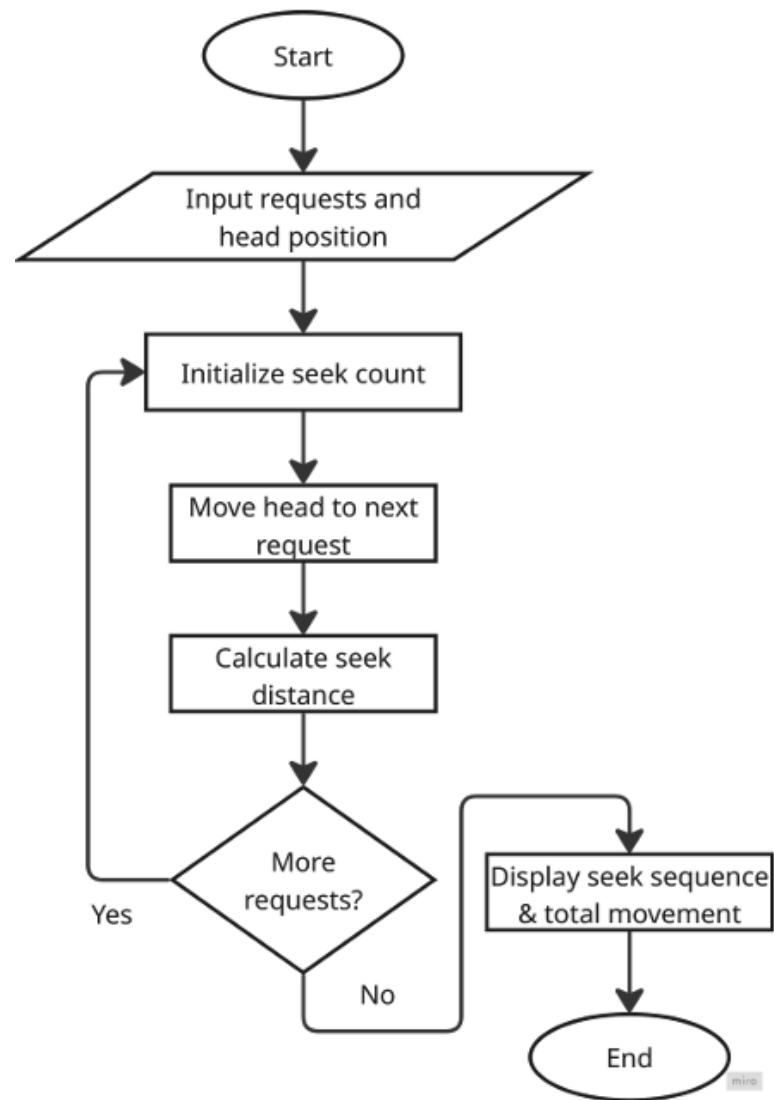
- Reduces unnecessary movement of the disk head.
- Decreases disk seek time and improves access speed.
- Improves overall disk I/O performance and efficiency.
- Helps manage multiple pending disk requests systematically.
- Balances performance and fairness among different requests.

First-Come, First-Served (FCFS)

First-Come, First-Served (FCFS) is a disk scheduling algorithm in which requests are serviced in the same order in which they arrive. It does not rearrange the pending request queue and therefore has a simple implementation.

Steps:

1. Start from the current head position.
2. Select the first request in the queue.
3. Move the head to that request and service it.
4. Select the next request in the original order.
5. Repeat until all requests are serviced.
6. Calculate the total head movement.



Advantages:

- Simple and easy to implement.
- Requests are serviced in their arrival order.
- Provides straightforward and predictable scheduling.

Disadvantages:

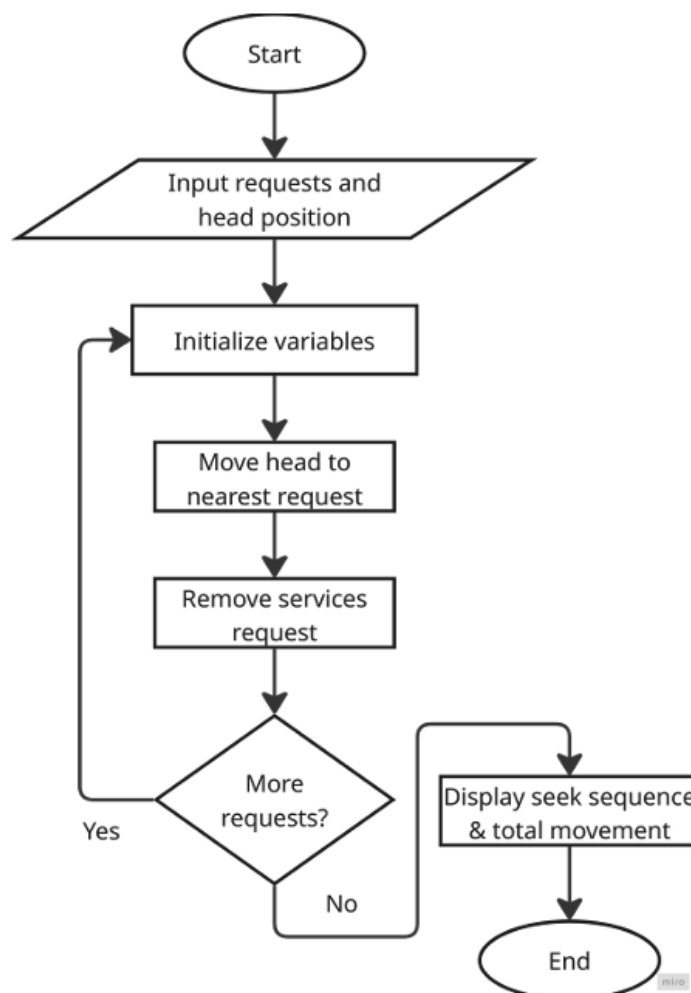
- May result in large head movement.
- Does not optimize seek time.
- Can provide poor performance when requests are located far apart.

Shortest Seek Time First (SSTF)

Shortest Seek Time First (SSTF) is a disk scheduling algorithm that selects the pending request closest to the current disk head position. Its main objective is to reduce the immediate distance the head has to travel for each request.

Steps:

1. Start from the current head position.
2. Calculate the distance to every pending request.
3. Select the request with the shortest distance.
4. Move the head to that request and service it.
5. Repeat the process until all requests are completed.



Advantages:

- Usually reduces head movement compared with FCFS.
- Can provide better seek performance.
- Gives priority to nearby requests.

Disadvantages:

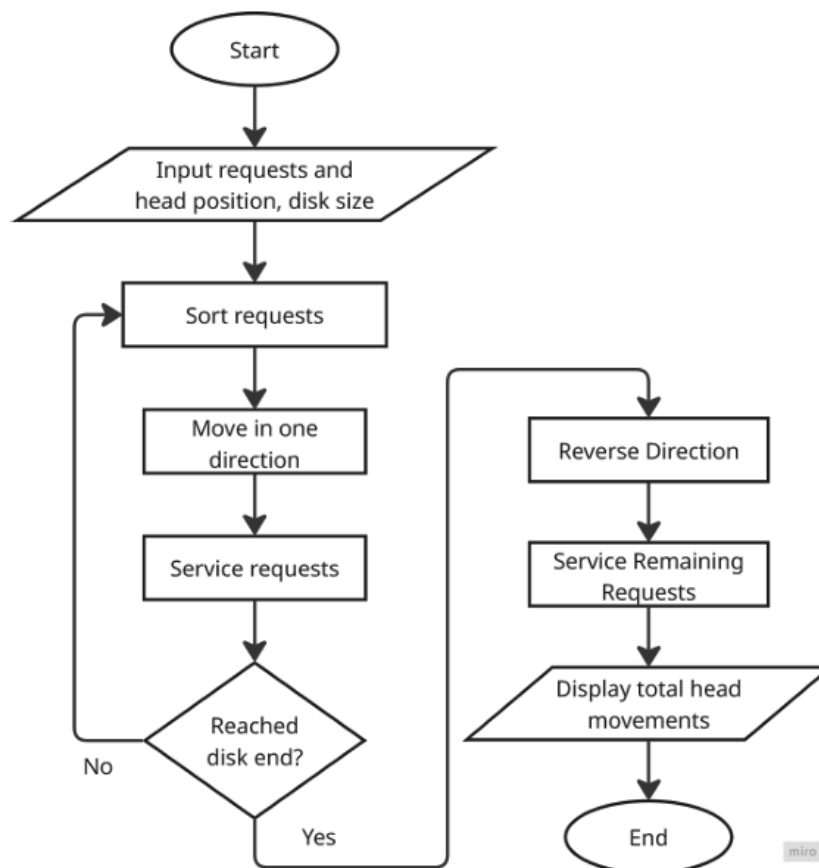
- Requests located far away may experience long waiting times.
- Starvation may occur if closer requests continue to arrive.
- Distance calculations are required before selecting each request.

SCAN

SCAN is a disk scheduling algorithm in which the disk head moves in one direction while servicing requests and then reverses direction after reaching the end of the disk. It is commonly known as the elevator algorithm because its movement resembles an elevator moving between floors.

Steps:

1. Start from the current head position.
2. Select a direction of movement.
3. Service requests encountered in that direction.
4. Continue toward the end of the disk.
5. Reverse the direction after reaching the end.
6. Service the remaining requests.



Advantages:

- Provides a more organized service pattern.
- Can provide more uniform waiting times than FCFS.
- Reduces unnecessary random movement.

Disadvantages:

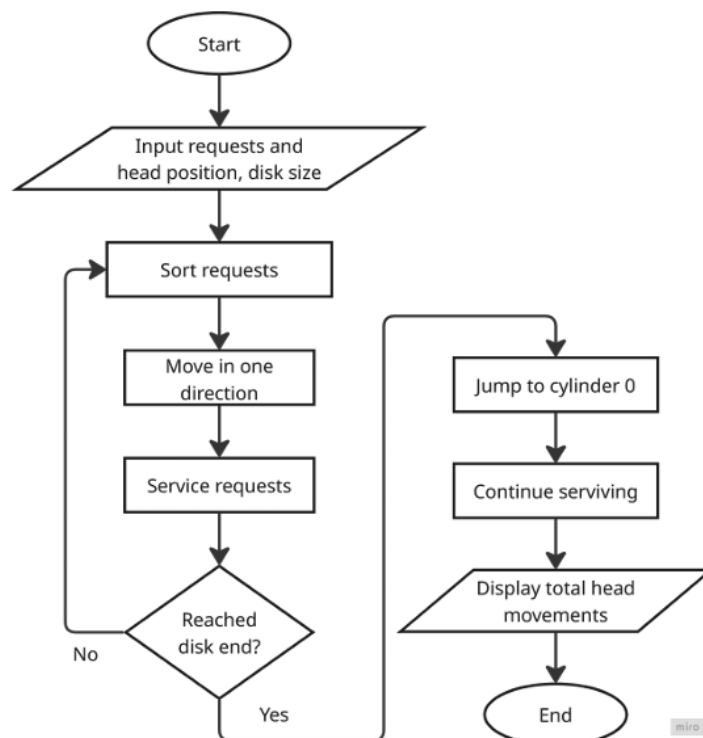
- The head may travel to the end even when no request exists there.
- Requests may wait until the head reverses direction.
- Performance depends on the initial direction of movement.

C-SCAN

C-SCAN, or Circular SCAN, is a disk scheduling algorithm that moves the disk head in only one direction while servicing requests. When the head reaches one end of the disk, it returns to the opposite end and continues servicing requests in the same direction.

Steps:

1. Start from the current head position.
2. Move in the selected direction.
3. Service requests encountered along the way.
4. Continue until the end of the disk is reached.
5. Move the head to the opposite end.
6. Continue servicing requests in the original direction.



Advantages:

- Provides a relatively uniform waiting time.
- Services requests in a consistent direction.
- Suitable when a more uniform service pattern is required.

Disadvantages:

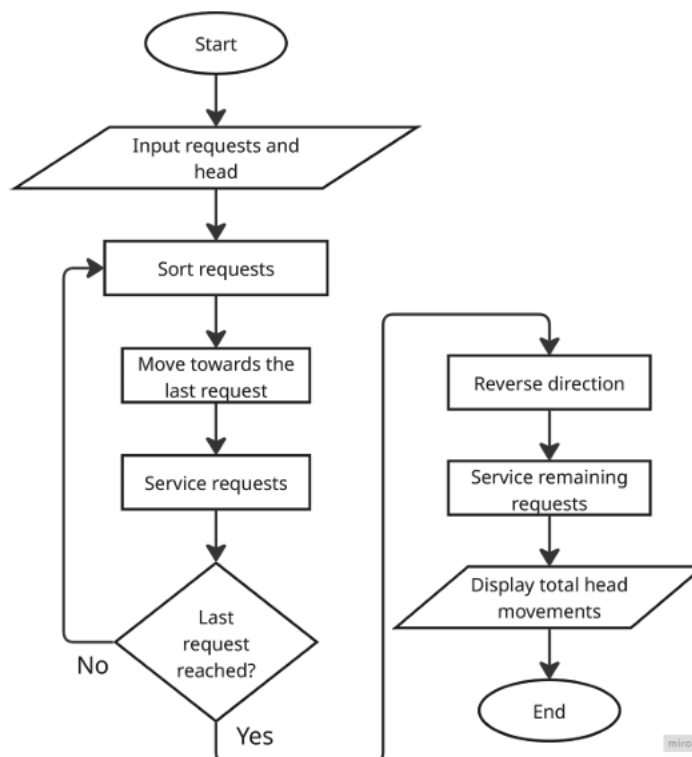
- The return movement does not service requests.
- It can result in additional head movement.
- May require more movement than LOOK or C-LOOK for some request patterns.

LOOK

LOOK is a variation of the SCAN disk scheduling algorithm. It moves the disk head in one direction and services requests along the way, but unlike SCAN, it does not necessarily move to the physical end of the disk.

Steps:

1. Start from the current head position.
2. Select a direction.
3. Service requests in that direction.
4. Stop when there are no more pending requests in that direction.
5. Reverse the direction.
6. Service the remaining requests.



Advantages:

- Avoids unnecessary movement to the disk boundary.
- Can reduce total head movement compared with SCAN.
- Maintains an organized service sequence.

Disadvantages:

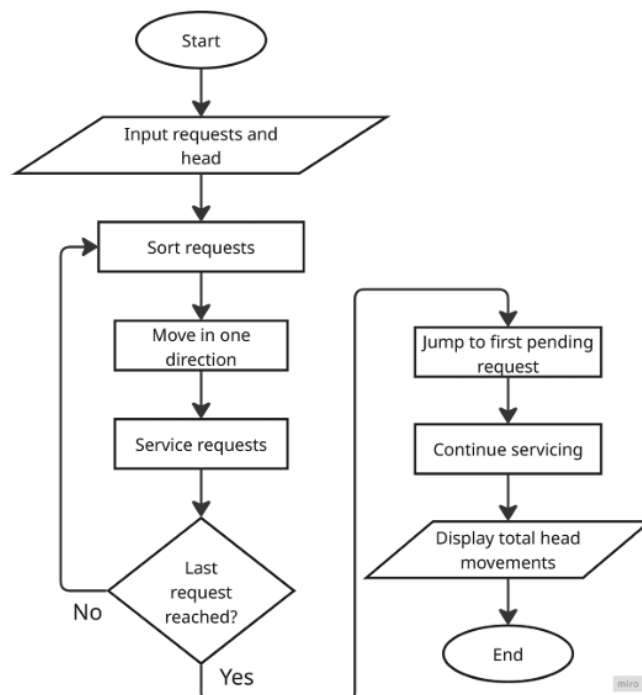
- Requests may still have to wait for a complete sweep.
- Performance depends on the distribution of requests.
- Initial direction can affect the service order and head movement.

C-LOOK

C-LOOK is a circular version of the LOOK disk scheduling algorithm. The disk head moves in one direction and services all requests encountered in that direction. When it reaches the last pending request in that direction, it moves to the first pending request at the opposite end and continues in the same direction.

Steps:

1. Start from the current head position.
2. Move in the selected direction.
3. Service requests encountered along the way.
4. Stop at the last pending request in that direction.
5. Move to the first pending request at the opposite end.
6. Continue in the original direction.



Advantages:

- Avoids unnecessary movement to the physical disk boundaries.
- Provides a circular and organized service pattern.
- Can reduce head movement compared with C-SCAN.

Disadvantages:

- The movement to the opposite side can still be significant.
- Requests may have to wait for the circular movement.
- Performance depends on request distribution and initial direction.

Source Code

1. First Come First Serve (FCFS)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(request) / sizeof(request[0]);
    int head = 53;
    int totalHeadMovement = 0;
    int i;

    printf("Disk Request Queue: ");
    for (i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }

    printf("\nInitial Head Position: %d", head);
    printf("\n\nHead Movement:\n");
    printf("%d", head);

    for (i = 0; i < n; i++) {
        totalHeadMovement += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalHeadMovement);

    printf("Average Seek Length = %.2f cylinders\n", (float)totalHeadMovement / n);

    return 0;
}
```

2. Shortest Seek Time First (SSTF)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(request)/sizeof(request[0]);
    int head = 53;
    int totalHeadMovement = 0;
    int completed[8] = {0};
    int i, j, closest, distance, minDistance;

    printf("Disk Request Queue: ");
    for (i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }
    printf("\nInitial Head Position: %d\n", head);
    printf("\nHead Movement:\n");
    printf("%d", head);

    for (i = 0; i < n; i++) {
        minDistance = 201;
        closest = -1;

        for (j = 0; j < n; j++) {
            if (completed[j] == 0) {
                distance = abs(head - request[j]);

                if (distance < minDistance) {
                    minDistance = distance;
                    closest = j;
                }
            }
        }
    }
}
```

```
    }  
    completed[closest] = 1;  
    totalHeadMovement += minDistance;  
    head = request[closest];  
    printf(" -> %d", head);  
}  
printf("\n\nTotal Head Movement = %d cylinders\n", totalHeadMovement);  
  
return 0;  
}
```

3. SCAN (Elevator)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(request)/sizeof(request[0]);
    int head = 53;
    int totalHeadMovement = 0;
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (request[i] > request[j]) {
                temp = request[i];
                request[i] = request[j];
                request[j] = temp;
            }
        }
    }

    printf("Disk Request Queue: ");
    for (i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }

    printf("\nInitial Head Position: %d", head);
    printf("\nDirection: Towards 200");

    printf("\n\nHead Movement:\n");
    printf("%d", head);

    for (i = 0; i < n; i++) {
```

```

    if (request[i] >= head) {
        totalHeadMovement += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
}

totalHeadMovement += 200 - head;
head = 200;
printf(" -> %d", head);

// Reverse direction and service remaining requests
for (i = n - 1; i >= 0; i--) {
    if (request[i] < head && request[i] < 53) {
        totalHeadMovement += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
}

printf("\n\nTotal Head Movement = %d cylinders\n", totalHeadMovement);

return 0;
}

```

4. CSCAN

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(request)/sizeof(request[0]);
    int head = 53;
    int totalHeadMovement = 0;
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (request[i] > request[j]) {
                temp = request[i];
                request[i] = request[j];
                request[j] = temp;
            }
        }
    }

    printf("Disk Request Queue: ");
    for (i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }

    printf("\nInitial Head Position: %d", head);
    printf("\nDirection: Towards 200");

    printf("\n\nHead Movement:\n");
    printf("%d", head);

    for (i = 0; i < n; i++) {
```

```

        if (request[i] >= head) {
            totalHeadMovement += abs(head - request[i]);
            head = request[i];
            printf(" -> %d", head);
        }
    }

    totalHeadMovement += 200 - head;
    head = 200;
    printf(" -> %d", head);

    // Jump from 200 to 0
    totalHeadMovement += 200;
    head = 0;
    printf(" -> %d", head);

    for (i = 0; i < n; i++) {
        if (request[i] < 53) {
            totalHeadMovement += abs(head - request[i]);
            head = request[i];
            printf(" -> %d", head);
        }
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalHeadMovement);

    return 0;
}

```

5. LOOK

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(request) / sizeof(request[0]);

    int head = 53;
    int totalHeadMovement = 0;
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (request[i] > request[j]) {
                temp = request[i];
                request[i] = request[j];
                request[j] = temp;
            }
        }
    }

    printf("Disk Request Queue: ");
    for (i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }

    printf("\nInitial Head Position: %d", head);
    printf("\nDirection: Towards 200");

    printf("\n\nHead Movement:\n");
    printf("%d", head);

    for (i = 0; i < n; i++) {
```

```

    if (request[i] >= head) {
        totalHeadMovement += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
}

for (i = n - 1; i >= 0; i--) {
    if (request[i] < 53) {
        totalHeadMovement += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
}

printf("\n\nTotal Head Movement = %d cylinders\n", totalHeadMovement);

return 0;
}

```

6. CLOOK

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(request) / sizeof(request[0]);

    int head = 53;
    int totalHeadMovement = 0;
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (request[i] > request[j]) {
                temp = request[i];
                request[i] = request[j];
                request[j] = temp;
            }
        }
    }

    printf("Disk Request Queue: ");
    for (i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }

    printf("\nInitial Head Position: %d", head);
    printf("\nDirection: Towards 200");

    printf("\n\nHead Movement:\n");
    printf("%d", head);
```

```

for (i = 0; i < n; i++) {
    if (request[i] >= head) {
        totalHeadMovement += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
}

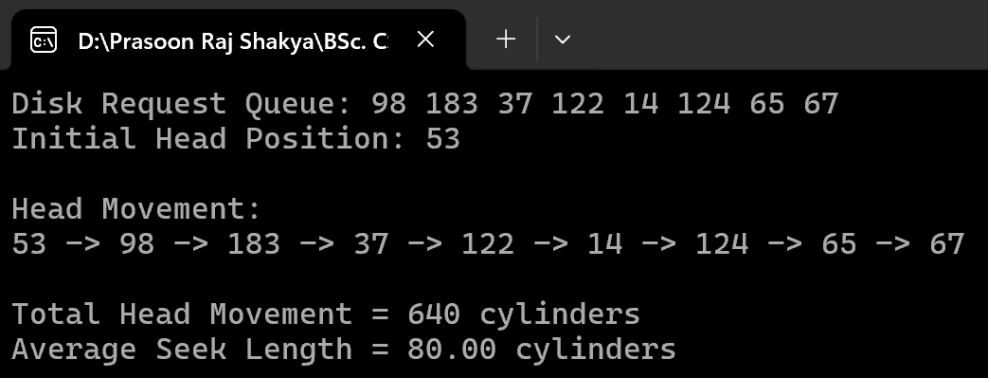
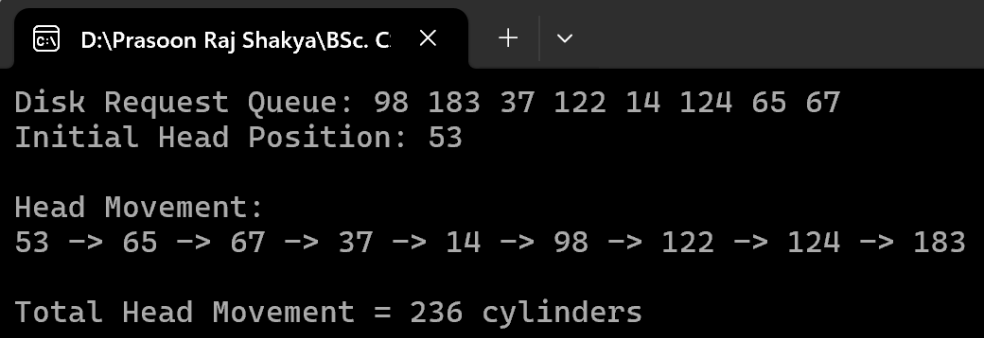
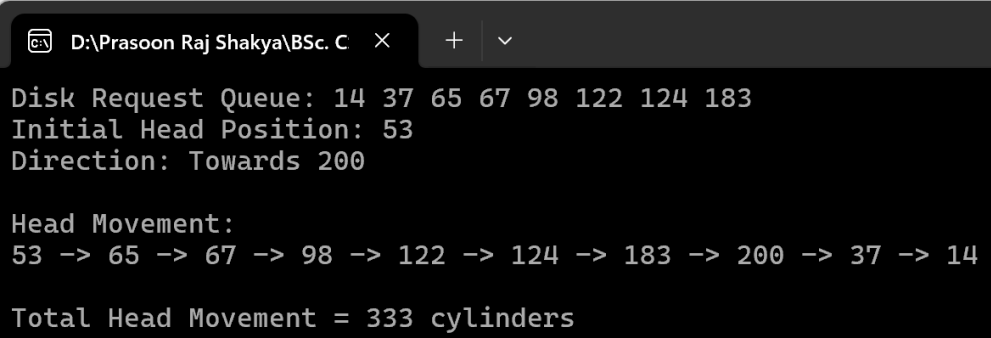
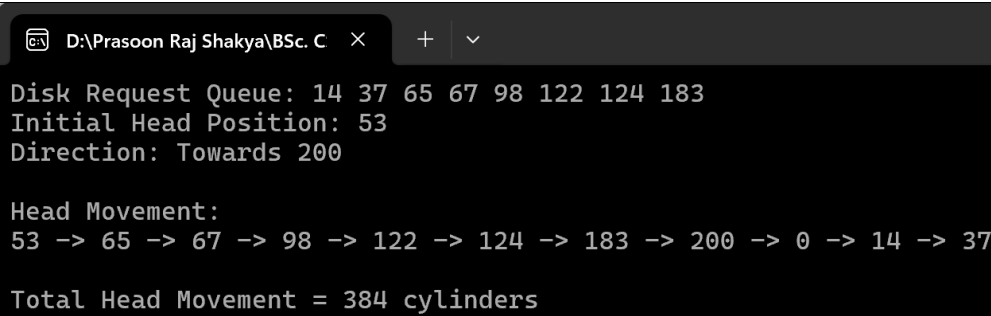
for (i = 0; i < n; i++) {
    if (request[i] < 53) {
        if (head != request[i]) {
            totalHeadMovement += abs(head - request[i]);
            head = request[i];
            printf(" -> %d", head);
        }
    }
}

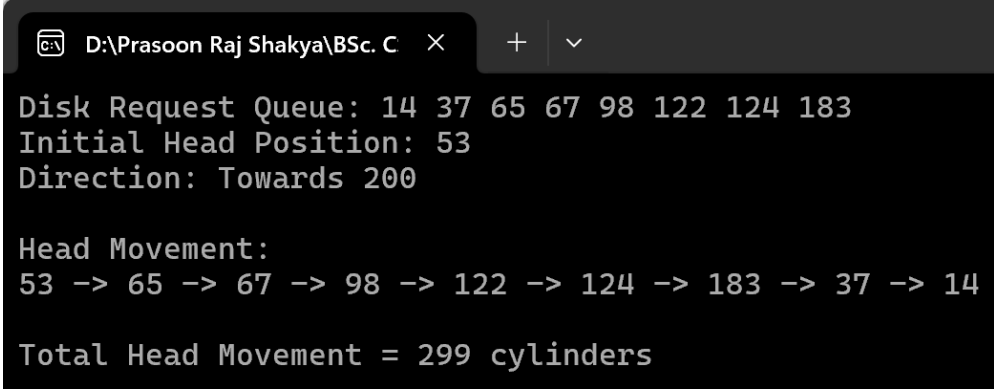
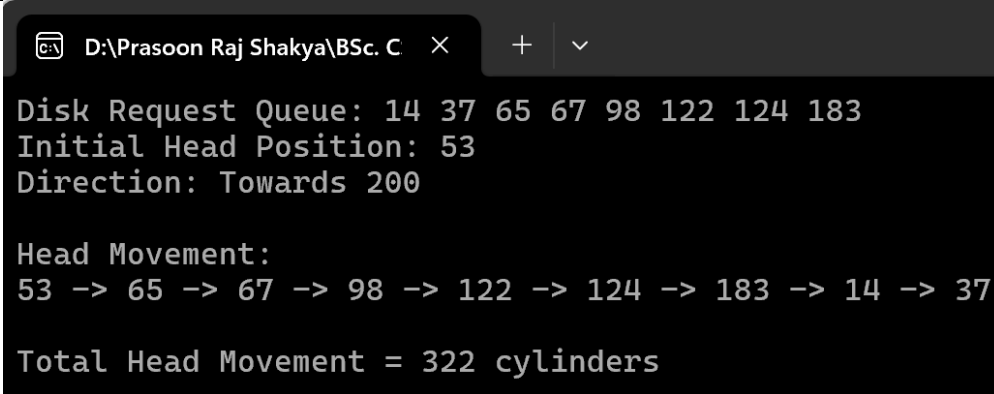
printf("\n\nTotal Head Movement = %d cylinders\n", totalHeadMovement);

return 0;
}

```

Output

1. FCFS	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Disk Request Queue: 98 183 37 122 14 124 65 67 Initial Head Position: 53 Head Movement: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67 Total Head Movement = 640 cylinders Average Seek Length = 80.00 cylinders </pre>
2. SSTF	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Disk Request Queue: 98 183 37 122 14 124 65 67 Initial Head Position: 53 Head Movement: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183 Total Head Movement = 236 cylinders </pre>
3. SCAN	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Disk Request Queue: 14 37 65 67 98 122 124 183 Initial Head Position: 53 Direction: Towards 200 Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 200 -> 37 -> 14 Total Head Movement = 333 cylinders </pre>
4. CSCAN	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Disk Request Queue: 14 37 65 67 98 122 124 183 Initial Head Position: 53 Direction: Towards 200 Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 200 -> 0 -> 14 -> 37 Total Head Movement = 384 cylinders </pre>

5. LOOK	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Disk Request Queue: 14 37 65 67 98 122 124 183 Initial Head Position: 53 Direction: Towards 200 Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14 Total Head Movement = 299 cylinders </pre>
6. CLOOK	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Disk Request Queue: 14 37 65 67 98 122 124 183 Initial Head Position: 53 Direction: Towards 200 Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> 37 Total Head Movement = 322 cylinders </pre>

Conclusion

In this experiment, the FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK disk scheduling algorithms were studied and implemented. For the given request queue and initial head position, the total head movements obtained were 640 tracks for FCFS, 236 tracks for SSTF, 333 tracks for SCAN, 384 tracks for C-SCAN, 299 tracks for LOOK, and 322 tracks for C-LOOK. The results showed that each algorithm produced a different amount of head movement because each uses a different scheduling strategy. The experiment demonstrated how the choice of disk scheduling algorithm affects the movement of the disk head and overall disk access efficiency.